

PyFT: Dynamic Data Race Detection for Free-Threaded Python

Alvin Tsang

Abstract

PyFT (<https://github.com/Alvicorn/PyFT>) is a dynamic data race detector for free-threaded Python 3.14, a runtime environment in which the removal of the Global Interpreter Lock (GIL) exposes programs to true concurrent memory access. The tool implements the VerifiedFT algorithm, adapting its vector-clock-based happens-before analysis to Python's instrumentation model via the `sys.monitoring` API. We describe the architecture of PyFT, the key design decision that made zero-patch instrumentation possible and future works.

1. Introduction

The Global Interpreter Lock (GIL) has been a topic of contention for CPython. By serializing bytecode execution across threads, concurrent loads would be safe by default. In the past, developers had to use libraries like `multiprocessing` and `asyncio` to get around the GIL to make their programs run concurrently.

Python Enhancement Proposal (PEP) 703 removes the GIL. In free-threaded Python, threads execute truly in parallel with real performance gains on CPU-bounded workloads. This also means that shared objects that were previously implicitly protected by the GIL are now exposed to concurrent access with no safety net.

With nearly over 20 years of Python3 code written with the assumption of the GIL, enormous amounts of existing Python code implicitly rely on the GIL to provide mutual exclusion. This poses an immediate and practical problem for projects that want to port over to free-threaded Python. For example, a dictionary accessed by multiple threads will never raise data race issues in GIL Python, not because the programmer wrote correct synchronization, but because the GIL makes individual operations appear atomic. Under free-threading, those programs may exhibit data races, corrupted state and non-deterministic crashes, all of which are notoriously difficult to reproduce and diagnose.

FasTrack [1] and its formally verified successor, VerifiedFT [2], provide theoretically optimal race detection algorithms. Other programming languages like C and C++ have tools like ThreadSanitizer [3] for race detection, but Python has no equivalent, as the GIL renders such tools unnecessary.

PyFT is a dynamic data race detector for free-threaded Python 3.14 and implements the VerifiedFT algorithm. Rather than modifying CPython or requiring source transformations, PyFT uses the `sys.monitoring` API to observe synchronization events at runtime. Memory accesses are tracked through a proxy-based wrapper that reports reads and writes to the detection engine on every attribute access. The result is a tool that requires no changes to the Python interpreter and no modification of the libraries it observes.

2. Background

2.1. Happens-Before and Data Races

A data race occurs when two threads access the same memory location concurrently, at least one of the accesses is a write, and there is no synchronization ordering them. A happens-before relation (HB) provides a partial order over events in a concurrent execution. For example, if event A happens-before event B, then A must complete before B begins. Synchronization operations establish these edges:

- Lock release happens-before the subsequent acquire of that lock
- Thread fork happens-before the child's first event
- Thread join happens-before the joiner's next event

Two accesses are concurrent, and therefore potentially racing, if neither happens-before the other.

Vector clocks are the standard mechanism for tracking happens-before at runtime [7]. Each thread maintains a vector clock C : a map from thread IDs to logical timestamps. On each event by thread t , $C[t]$ is

incremented. Lock release records a snapshot of the releasing thread's clock, and lock acquire merges that snapshot into the acquiring thread's clock. A cross-thread HB check then reduces to a scalar comparison.

2.2 FastTrack and VerifiedFT

Naive vector-clock race detection is $O(n)$ per access in the number of threads, because the full clock must be examined. FastTrack [1] observed that in typical programs, reads are dominated by single-reader patterns. It replaces the read vector clock with an epoch, a $(tid, clock)$ pair, reducing read checks to $O(1)$. VerifiedFT [2] extends FastTrack with a machine-checked correctness proof using the Coq proof assistant, and adds formal verification of the read-to-write transition, which contains a subtle case that was incorrect in earlier formulations.

Each variable x in VerifiedFT maintains two shadow fields, a write epoch $W_x = (tid, clock)$ representing the last write, and a read state $R_x \in \{\text{Bottom}, \text{Epoch}(tid, clock), \text{VC}(vc)\}$ representing prior reads. The write rule checks the incoming write against both W_x and R_x , and the read rule checks against W_x alone. Both rules update the shadow state atomically and report a race if the new access is unordered with existing epochs.

2.3. sys.monitoring

Python 3.12 introduced `sys.monitoring` (PEP 669), a lightweight event observation API. Unlike the older `sys.settrace`, which fires a callback on every source line and every function call regardless of interest, `sys.monitoring` allows precise registration of specific event types: `CALL`, `PY_RETURN`, `C_RETURN`, `PY_START`, and others. Callbacks fire only for the registered types, and a callback can return `sys.monitoring.DISABLE` to permanently disable itself at that specific code location, allowing the tool to self-tune by silencing sites that are not synchronization primitives after the first inspection.

3. Related Work

ThreadSanitizer [3] is a compile-time instrumentation pass for C, C++, and Go that detects data races at runtime using shadow memory and a happens-before algorithm. Integrated with LLVM and GCC, it reports 5–15× slowdown in practice. It does not apply to Python as it instruments native machine code and

cannot observe Python-level synchronization primitives, which are implemented as Python method calls rather than memory fences.

FastTrack [1] demonstrated that race detection can be made efficient by observing that the vast majority of variable accesses follow a single-reader pattern. By storing an epoch instead of a full vector clock for reads, the common-case check becomes $O(1)$. VerifiedFT [2] proved FastTrack correct in Coq and identified an error in the read-to-write transition. PyFT implements VerifiedFT's read and write rules.

Within Python, available tooling is limited. The built-in `pdb` debugger is fundamentally single-threaded: it installs a `sys.settrace` callback that acquires the GIL and does not expose concurrent thread interactions. `py-spy` is a sampling profiler that captures thread-state snapshots but cannot detect races or deadlocks. Neither was designed for free-threaded execution.

DynaPyt is a Python dynamic analysis framework built over `sys.settrace` and AST instrumentation. It was considered as infrastructure for PyFT early in development, but was not adopted because of the overhead it introduces. Additionally, its event model does not map cleanly to the lock-acquire/release and fork/join events required by VerifiedFT, and its design does not account for the reentrancy requirements of a concurrent race detector.

Modifying CPython directly was also considered. This would provide maximum control and a reduced overhead. However, it would have required deep knowledge of the CPython interpreter internals, would have produced a tool that breaks with each CPython release and can not be distributed to standard Python installations.

4. Design & Architecture

4.1. Overview

PyFT is organized around a central Engine that maintains the VerifiedFT shadow state and processes all synchronization events. Two instrumentation layers feed into the engine. SyncMonitor installs `sys.monitoring` callbacks to observe synchronization primitives (locks, barriers, events, condition variables, thread start/join) without modifying them. AutoTracker patches `Thread.start` to observe the thread lifecycle. It establishes the fork happens-before

edge from parent to child before the child begins executing, and wraps the thread's run method to call `engine.thread_finish` on exit. Memory accesses reach the engine through `TrackedProxy`, a transparent attribute-access wrapper that reports reads and writes on every get and set.

4.2. Shadow State

The Engine maintains a `ShadowMap` that maps `object_id` and `attribute_name` pairs to `VarState` objects. Each `VarState` holds a `write_epoch` (either an `Epoch(tid, clock)` or `Bottom`) and a `read_state` (`ReadBottom`, `ReadEpoch`, or `ReadVC`). Both fields are protected by a per-`VarState` lock.

4.3. Thread Registry

The `ThreadRegistry` maps thread IDs to `ThreadState` objects. Each `ThreadState` holds the thread's current vector clock, implemented as a dict mapping thread IDs to integer timestamps.

Thread identity is represented as the Python object ID of the current thread, rather than `threading.get_ident()`, which returns an OS thread ID. OS thread IDs are small integers assigned by the kernel and are reused when threads are created and destroyed within the same session. Conversely, Python object IDs are unique for the lifetime of the object. Since `Thread` objects outlive their OS threads, they provide a stable identity for the duration of a PyFT session.

4.3. VerifiedFT Rules

On `engine.write(obj, attr)`, the engine ticks the current thread's clock, takes a snapshot, and calls `var.check_write(tid, snapshot)`. Inside `check_write`, with `VarState._lock` held:

- The prior `write_epoch` and `read_state` are captured atomically before any updates.
- Write vs. prior write: if `write_epoch.tid` $\neq$ current and `write_epoch` does not happens-before current VC $\rightarrow$ write-write race.
- Write vs. prior reads: if any reader's epoch does not happens-before the current VC $\rightarrow$ read-write race.
- `write_epoch` is updated to (`current_tid`, `current_clock`); `read_state` is reset to `Bottom`.

On `engine.read(obj, attr)`, the engine ticks and calls `var.check_read(tid, snapshot)`. Inside, with `VarState._lock` held: if `write_epoch` is `Bottom`, or the same thread, or happens-before the current VC, the access is safe, and `read_state` is updated. Otherwise, a write-read race is reported with the prior `write_epoch`.

4.4. Synchronization Observation

`SyncMonitor` registers callbacks for four `sys.monitoring` event types. `CALL` fires before any Python callable and is used to classify synchronization operations. If the callable is a method of a recognized type (`Lock`, `RLock`, `Semaphore`, `Condition`, `Event`, `Barrier`, `Thread`) with a recognized method name (`acquire`, `release`, `wait`, `notify`, `start`, `join`), a `SyncCall` record is produced. Events that fire on call alone (`release`, `Event.set`, `Condition.notify`) dispatch to the engine immediately. Events that need return-value correlation (`acquire`, `join`, `Event.wait`, `Barrier.wait`) push a record onto a per-thread pending stack.

`C_RETURN` fires when a C-implemented function returns, `Lock.acquire` is C-level in CPython, so its result arrives here. `PY_RETURN` handles Python-implemented returns. `PY_START` fires on the first bytecode of a new thread's frame and is used to detect threads that bypass `Thread.start` (e.g., threads created by C extensions) and to correlate pending fork records.

5. Evaluation

5.1. Unit and Integration Tests

A suite of unit tests directly exercises the VerifiedFT rules, vector clock operations, and the synchronization classification logic of `SyncMonitor` in isolation. Integration tests exercise the happens-before properties that PyFT must respect end-to-end.

5.2. DataRaceBench

`DataRaceBench` [4] is a benchmark suite originally developed for C/C++ OpenMP programs to systematically evaluate data race detection tools. Each benchmark is labelled "yes" (contains a known data race) or "no" (correctly synchronized). The benchmarks cover a range of concurrency patterns: write-write races, read-write races, races through aliased pointers or list elements, lock-protected access, reductions, and races within nested loops.

100 benchmarks were manually translated from C to Python 3.14, preserving the synchronization structure and race patterns of the originals. Translation required adapting C-specific idioms (pointer aliasing, struct field access via pointer arithmetic) to Python’s attribute-access model. Pointer aliasing benchmarks, for example, become two references to the same Python object, both accessed through TrackedProxy wrappers that share the same underlying object identity.

PyFT correctly classifies all 100 benchmarks: races are detected on every “yes” case, and no race is reported on any “no” case. No false positives or false negatives were observed.

5.2. Performance

Instrumentation at the Python level carries inherent overhead that warrants careful methodology to accurately measure, but this was outside the scope of this project, as significant effort was put into design and implementation. What follows is a qualitative analysis of where overhead is incurred and how it might be characterized.

The dominant cost is per-access overhead. Every attribute read or write on a tracked object invokes an Engine read or write. These operations perform a thread-local lookup, a clock tick, a lock acquisition on the per-variable VarState, a vector clock comparison, and a conditional shadow state update. In a program where tracked attributes are accessed in tight loops, this overhead would be applied to every iteration.

Secondary cost comes from sys.monitoring callbacks. The CALL event fires before every Python function call in the process. The self-disable mechanism reduces this to zero for sites that are never synchronization primitives after their first invocation, but in large programs with many unique call sites, the initial classification pass has non-trivial cost. C_RETURN similarly fires on every C function return.

In free-threaded Python, all of the above runs without a GIL, meaning internal PyFT locks can become contention points under high thread counts. Unlike GIL-Python, where only one thread executes at a time anyway, multiple threads in free-threaded Python can genuinely compete for these locks.

For comparison, ThreadSanitizer, which instruments compiled native code with essentially zero interpreter

overhead, reports 5-15x slowdown. Python-level instrumentation adds the interpreted function-call overhead of the engine itself on top of that. A realistic estimate for PyFT in its current form would be 20-100x slowdown on access-intensive workloads, though only a systematic benchmarking study can substantiate this.

6. Limitations and Future Work

The 100 DRB benchmarks were translated manually. Each required individual inspection and adaptation from C to Python. The right long-term solution is an automated loader and AST transformer that converts C/C++ DRB benchmarks to Python equivalents, enabling the full DataRaceBench suite of over 1,000 benchmarks to be evaluated without manual effort.

Instrumentation coverage is currently limited to explicitly tracked objects. Users must wrap objects with TrackedProxy or use the make_tracked_class decorator. There is no mechanism to automatically detect that a plain Python object is being shared across threads and begin tracking it. A complete solution would require either import-hook-based AST transformation (inserting engine.read/engine.write calls at every attribute access site at import time) or a C-level implementation hooking into Python’s object slot protocol directly.

Deadlock detection was identified in the project proposal as a goal. It was not implemented within the project timeline. The existing SyncMonitor infrastructure provides a natural foundation since it already observes every lock acquire and release. A resource-allocation graph could detect cycles in real time. This is a straightforward extension that does not require changes to the core VerifiedFT logic.

The VectorClock and ThreadState implementations are not fully thread-safe in free-threaded Python. Individual dict operations are atomic in CPython’s implementation even without the GIL, but compound operations like clock increment (read-then-write on the same key) are not. Under high thread counts with many concurrent writes, clock values could be corrupted. A production implementation would use atomic compare-and-swap operations for clock updates, or add per-ThreadState locks for clock mutations.

Finally, PyFT currently has no complete integration with existing Python development workflows. An

ideal deployment would allow users to run their programs under pyft with a single command (e.g., `python -m pyft myscript.py`) and receive structured race reports. This requires an import hook that automatically wraps user-defined classes and a report formatter that maps VarState epochs to human-readable source locations.

References

- [1] C. Flanagan and S. N. Freund. FastTrack: Efficient and Precise Dynamic Race Detection. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2009.
- [2] J. R. Wilcox and S. N. Freund. VerifiedFT: A Verified, High-Performance Precise Dynamic Race Detector. ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), 2018.
- [3] K. Serebryany and T. Iskhodzhanov. ThreadSanitizer: Data Race Detection in Practice. Workshop on Binary Instrumentation and Applications (WBIA), 2009.
- [4] C. Liao, B. R. de Supinski, M. Schulz, and D. Quinlan. DataRaceBench: A Benchmark Suite for Systematic Evaluation of Data Race Detection Tools. International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2017.
- [5] S. Naschert. PEP 703 — Making the Global Interpreter Lock Optional in CPython. Python Enhancement Proposals, 2023.
- [6] M. Shannon. PEP 669 — Low Impact Monitoring for CPython. Python Enhancement Proposals, 2022.
- [7] L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 1978.